The criterion-space search, step by step

what each line of the algorithm does, and why it is that line

Abstract

This is a companion to the main note: it takes the algorithm one line at a time and says what the line computes, what it costs, what breaks without it, and which of the package's tests pins it. It assumes only the problem statement of Section 1; the measurements and the comparison with other methods live in the main note, and the two worked examples live in the companion *Worked examples*.

Contents

1 The setting, in one page **1**

2 The one object the algorithm manipulates **1**
2.1 e_k: the linear form that carries a ratio's sign 1
2.2 A box 2

3 The efficiency test **2**

4 The split **2**

5 The algorithm, line by line **3**
5.1 Line 5 — best bound first 3
5.2 Line 6 — stopping on a hopeless best bound 4
5.3 Line 9 — the sub-problem 4
5.4 Line 11 — dropping a box whole 4
5.5 Line 14 — reusing a point already proved efficient 4
5.6 Line 19 — when the maximiser is already efficient 4
5.7 Line 21 — when it is not 5
5.8 Line 25 — the sub-problem Q 5
5.9 Line 28 — the split 5

6 What is checked, and how **5**

1 The setting, in one page

Let

$$(P_D) \qquad \text{“max”} \quad Z_k(x) = \frac{c_k^\top x + a_k}{d_k^\top x + b_k}, \quad k = 1, \dots, p, \qquad x \in D = \{\, x \in \mathbb{Z}_+^n : Ax \leqslant b \,\}, \tag{1}$$

with D non-empty and bounded and every denominator positive on it. A point $x \in D$ is *efficient* when no $y \in D$ has $Z(y) \geqslant Z(x)$ with one strict inequality; $E(P_D)$ is the set of them.

Given the decision maker's own criterion $\Phi(x) = (U^\top x + \alpha)/(V^\top x + \beta)$, the problem is

$$(P_E) \qquad \max \ \Phi(x) \quad \text{s.t.} \quad x \in E(P_D). \tag{2}$$

Three facts drive everything below.

$E(P_D)$ **has no description.** It is a union of faces of D, non-convex, and not given by inequalities. So (P_E) cannot be handed to a solver; a method has to *generate* candidates and prove the rest away.

Φ **is a function of** x**, not of** $Z(x)$**.** Two feasible points with the same criterion vector can have different Φ. Enumerating non-dominated vectors and scoring one representative of each is therefore wrong, and this is what makes the criterion-space transfer non-trivial.

Efficiency is decidable by one integer program. Theorem 1's test, Section 3.

2 The one object the algorithm manipulates

2.1 e_k: the linear form that carries a ratio's sign

Write, for a reference point $a \in D$,

$$e_k(x;a) \;=\; D_k(a)\,D_k(x)\,\big(Z_k(x)-Z_k(a)\big) \;=\; \big(D_k(a)c_k-N_k(a)d_k\big)^{\top}x+\big(D_k(a)a_k-N_k(a)b_k\big), \quad (3)$$

where N_k, D_k are the numerator and denominator of Z_k. Two properties, and the whole method rests on them:

1. $e_k(\cdot\,;a)$ is **linear in** x — the right-hand side of (3) is an affine function — even though Z_k is a ratio.
2. Both denominators being positive, e_k carries the **sign** of $Z_k(x) - Z_k(a)$. With integer data it is integer-valued on integer points, so

$$Z_k(x) > Z_k(a) \iff e_k(x;a) \geqslant 1, \qquad Z_k(x) \leqslant Z_k(a) \iff e_k(x;a) \leqslant 0. \quad (4)$$

Why not just write $l_k \leqslant Z_k(x) \leqslant u_k$**?** Because Z_k is a ratio, its values are rational, and there is no smallest step between two of them to put in a strict inequality. Equation (4) replaces an estimated tolerance by an exact integral threshold, and it does so without special-casing: on a *linear* criterion ($d_k = 0$, $b_k = 1$) e_k collapses to $c_k^{\top}x - c_k^{\top}a$ and every formula below becomes the one written for linear criteria.

2.2 A box

A **box** is a region of criterion space, carried as a list of rows on x, each of the form coeffs$\cdot x \leqslant$ rhs, together with an upper bound on Φ over it inherited from its parent. The root box is all of $\mathbb{R}^p$ and carries no rows.

Every row a box ever receives is one of the two halves of (4), so a box is exactly an axis-aligned region of criterion space — which is what makes the pictures in the companion *Worked examples* faithful rather than illustrative.

3 The efficiency test

$$\theta(x^*) \;=\; \max \sum_{k=1}^{p} e_k(x;x^*) \quad \text{s.t.} \quad e_k(x;x^*) \geqslant 0 \;\; \forall k, \qquad x \in D. \quad (5)$$

x^* is feasible for it with value 0, so $\theta \geqslant 0$ always, and x^* is efficient exactly when $\theta = 0$. Every coefficient is an integer, so θ is an integer: the test is exact, with no tolerance, unlike a formulation written directly on the ratios.

Proposition 1 (Ecker & Kouada)**.** *With* linear *criteria, every optimal solution of* (5) *is itself efficient.*

Proof. With $d_k = 0$ the objective is $\sum_k (Z_k(x) - Z_k(x^*))$, a strictly positive weighted sum of Z up to a constant, maximised over $R = \{\, x \in D : Z(x) \geqslant Z(x^*) \,\}$. Let y^* be a maximiser and suppose $z \in D$ dominated it. Then $Z(z) \geqslant Z(y^*) \geqslant Z(x^*)$, so $z \in R$, and $\sum_k Z_k(z) > \sum_k Z_k(y^*)$ — contradicting optimality. The argument never assumes y^* unique, so it covers ties. □

This matters twice. It gives the algorithm its cut centre for free when the criteria are linear; and it *fails* for fractional criteria, where the k-th term of (5) carries a factor $D_k(x)$ that varies from point to point, the objective stops being monotone in Z, and a dominating point can score lower than a dominated one. There the maximiser only dominates, and a walk along the dominance chain is paid until a test returns $\theta = 0$.

4 The split

Proposition 2. *Let B be a box and $a \in D$. For $k = 1, \ldots, p$ put*

$$B_k \;=\; B \;\cap\; \{\, x : e_k(x; a) \geqslant 1 \,\} \;\cap\; \bigcap_{j<k} \{\, x : e_j(x; a) \leqslant 0 \,\}. \tag{6}$$

Then $B_1, \ldots, B_p$ are pairwise disjoint and $\bigcup_k B_k = B \setminus \{\, x : Z(x) \leqslant Z(a) \,\}$.

Proof. Let $x \in B$ with $Z(x) \not\leqslant Z(a)$. The set of indices where x strictly beats $Z(a)$ is non-empty; let k be its smallest element. By (4), $e_k(x; a) \geqslant 1$ and $e_j(x; a) \leqslant 0$ for $j < k$, so $x \in B_k$. Conversely a point of B_k has $e_k \geqslant 1$, hence $Z(x) \not\leqslant Z(a)$. Membership of B_k forces k to be exactly that smallest index, and a point has only one — so the B_k are disjoint. □

Each child is the parent's row list plus at most k further rows, all ordinary linear inequalities. **This is the whole difference from the decision-space method.** There the same truncation is a disjunction — "some criterion strictly improves" — which is not a system of linear inequalities and is modelled with p binaries and $p + 1$ big-M rows *per cut*, so every sub-problem is strictly larger than the last. Here the model never grows: a deep box carries a few more ordinary rows than a shallow one, and that is all.

5 The algorithm, line by line

Algorithm 1 Criterion-space search for (P_E)

1: $L \leftarrow \{\, \mathbb{R}^p \,\}$ ▷ the open list, a heap on the inherited bound
2: $(x_{\text{opt}}, \Phi_{\text{opt}}) \leftarrow$ a verified efficient seed, or $(\varnothing, -\infty)$
3: $\mathcal{S} \leftarrow \varnothing$ ▷ points already proved efficient
4: **while** $L \neq \varnothing$ **do**
5: $B \leftarrow$ the box of L with the largest inherited bound
6: **if** B carries a bound and it does not beat Φ_{opt} **then**
7: **stop**: no box still open can improve ▷ the answer is already proved
8: **end if**
9: solve $\max\{\, \Phi(x) : x \in D,\ \text{rows of } B \,\}$ with cutoff Φ_{opt}
10: **if** infeasible, or its value $\leqslant \Phi_{\text{opt}}$ **then**
11: **continue** ▷ B is discarded whole, never split
12: **end if**
13: $x_B \leftarrow$ its maximiser; B.bound $\leftarrow$ its value
14: **if** some $a \in \mathcal{S}$ dominates x_B **then**
15: $\tilde{x} \leftarrow$ the highest such a ▷ no efficiency test is paid
16: **else**
17: run the test (5) on x_B
18: **if** $\theta(x_B) = 0$ **then**
19: $\tilde{x} \leftarrow x_B$; update $(x_{\text{opt}}, \Phi_{\text{opt}})$ with $\Phi(x_B)$
▷ x_B maximises Φ on its whole slice already, so no Q is owed
20: **else**
21: $\tilde{x} \leftarrow$ an efficient point dominating x_B ▷ Proposition 1, or a walk
22: **end if**
23: **end if**
24: **if** $\tilde{x} \neq x_B$ **then**
25: solve $Q(\tilde{x}) = \max\{\, \Phi(x) : x \in D,\ Z(x) = Z(\tilde{x}) \,\}$; update the incumbent
26: **end if**
27: $\mathcal{S} \leftarrow \mathcal{S} \cup \{\tilde{x}\} \cup \{\text{the } Q \text{ maximiser}\}$
28: add to L the p children of B given by (6) around $\tilde{x}$
29: **end while**
30: **return** $(x_{\text{opt}}, \Phi_{\text{opt}})$, proved optimal

5.1 Line 5 — best bound first

The list is a heap ordered by *inherited* bound, descending; the root, and any box a hybrid hands over, carry no bound and rank first. Taking the most promising box first raises the incumbent early, which is what lets the rest be dropped whole at line 11.

5.2 Line 6 — stopping on a hopeless best bound

Because the heap is ordered by bound descending and the unbounded boxes rank first, by the time a *bounded* box reaches the top every box still open is bounded too. If this one cannot beat the incumbent, none of them can: the answer is already proved and the remaining list is busywork.

Measured before this line existed: 198 of 733 sub-problems were solved despite carrying such a bound, and on *every one* of the 18 instances that count equalled the number of boxes still open when the condition first fired. The waste is exactly a tail, never scattered — which is why one early exit is the whole fix rather than a filter on every box.

5.3 Line 9 — the sub-problem

D plus at most $2p$ ordinary rows, and **no binary variable anywhere**. The cutoff turns the branch and bound into the question "is there a point better than the incumbent, and if so which is best", which prunes most of the tree in the late boxes.

The child's region is the parent's plus a handful of rows, so its root relaxation starts from the parent's basis rather than from a phase I: the parent's basis stays a basis once the new rows' slacks join it, only those rows can be primal infeasible, and a dual restoration repairs them. A restoration that stalls falls back to the cold solve, so a warm start that does not work out costs time and never correctness.

5.4 Line 11 — dropping a box whole

The sub-problem maximises Φ over a *superset* of the efficient points whose criterion vector lies in B — it ignores efficiency entirely — so its value bounds all of them. A box whose bound does not beat the incumbent is therefore discarded together with every descendant it would have had.

This is the mechanism a better incumbent feeds, and it is why seeding the search is worth anything at all. The decision-space method has no counterpart: what it cuts is decided by the efficiency test on its own maximiser, so the same free optimum saves it *zero* iterations.

5.5 Line 14 — reusing a point already proved efficient

What the algorithm needs after line 9 is an efficient point to cut on. The test at line 17 produces one, and it is the single most expensive operation in the search — 52% of all simplex pivots, on 146 calls out of 806 sub-problems. But $\mathcal{S}$ already holds points *proved* efficient, and asking whether one of them dominates x_B is arithmetic. One does on 14% of the tests.

Taking the *highest* such point is not a detail: (6) removes $\{Z \leqslant Z(\tilde{x})\}$, so a higher centre removes more of the box. Skipping the test and cutting better compound, and the box count falls too.

The invariant this rests on. Every point in $\mathcal{S}$ is efficient. It receives the centres cut on and, beside each, the maximiser of Q — which shares its centre's criterion vector and is therefore efficient whenever the centre is. A single inefficient member would let the search cut on a dominated point and delete the true optimum, so this is pinned by a test rather than by this paragraph.

5.6 Line 19 — when the maximiser is already efficient

Then x_B is a genuine feasible point of (P_E) and its value is a valid incumbent. No Q is owed: x_B maximises Φ over the whole box, hence over the slice $\{Z = Z(x_B)\}$ inside it.

5.7 Line 21 — when it is not

x_B is dominated, so its value is **not** a valid incumbent — taking it would be the one way to lose the optimum, because the cutoff at line 9 would then discard boxes that could still beat the true answer. What the algorithm needs is an efficient point dominating x_B: Proposition 1 hands it over for linear criteria, and for fractional ones a walk along the dominance chain reaches one.

5.8 Line 25 — the sub-problem Q

The slice $\{Z = Z(\tilde{x})\}$ is entirely efficient — nothing can dominate a point of it without dominating $\tilde{x}$ — and it is about to be removed by the split. Since Φ is a function of x and not of $Z(x)$, its best value on that slice need not be attained at $\tilde{x}$, so it is computed and banked before the region goes.

This line is the one an implementation reading only the criterion vectors would omit, and omitting it loses optima silently: the answer stays plausible and is wrong only on instances where the slice is not a singleton.

5.9 Line 28 — the split

Proposition 2, with $\tilde{x}$ as centre. Termination follows: every pass removes at least the criterion vector $Z(\tilde{x})$ from the open region, and the non-dominated set of a bounded integer program is finite.

6 What is checked, and how

Several of the package's tests check *properties* rather than answers, because a method that silently lost efficient points would still agree with the optimum on favourable instances.

Table 1: The line, and what pins it.

line	claim	how it is checked
28	Proposition 2	for every centre and every feasible point: removed by the cut and in no child, or kept and in exactly one
6	the exit keeps the proof	the run must end with boxes still open and unsolved, stay proved optimal, and agree with an independent scan
14	$\mathcal{S}$ holds only efficient points	every remembered point is re-tested; 0 failures over 34
21	Proposition 1	349 witnesses of failed tests on linear criteria, none dominated; the shortcut returns exactly what the walk it replaces returns
9	the warm start is sound	a row added to a solved tableau, against rebuilding the model and solving it cold, on 120 random programs
seed	an efficient seed keeps the answer	seeded and unseeded runs agree and are both proved

That table pins what each line *does*. It says nothing about the hypotheses under which the lines are entitled to do it, and that turned out to be the gap: an audit found eleven places where a hypothesis stated in the text was never tested in the code. The pattern is worth stating here, because it is a pattern about pseudocode.

Every flag in the implementation that claimed something had been *proved* was computed from control flow. Line 6 is the clearest case: the box list running empty was reported as "the front is complete", which is sound exactly when D is bounded and every D_k is positive on it. Neither was established. The pseudocode is not wrong — those are its stated hypotheses — but "the loop finished" and "the hypotheses hold" are different propositions, and the implementation returned the first while reporting the second. Both failures are silent: on a feasible, unbounded D the enumeration returned an *empty front marked complete*, and on a criterion whose denominator changes sign it returned one vector as the whole front while a feasible point dominated it.

Table 2 lists what now enforces each hypothesis before the algorithm starts. They are cheap enough to leave on permanently.

The second half of the pattern is the more general lesson. Every parameter added to the implementation in order to *save* work — a precomputed variable box, a precomputed front, a set of

Table 2: The hypotheses, and what establishes them. Each was previously assumed; each failure was silent.

hypothesis	enforced by	cost
D bounded	one integer program, $\max \sum_j x_j$	one program per problem
$D_k(x) > 0$ on D	one program per criterion, minimising D_k	nothing for a linear criterion
a given variable box encloses D	n programs, compared against the box	n programs, cached on the problem
a given seed is efficient	the exact test of §3	the test the seed needed anyway
a given incumbent is feasible and realises its value	feasibility of the model, then Φ recomputed	arithmetic only
the returned point is efficient	the exact test, on exit	one test per run

seeds, an incumbent and its value — is a way for a caller to supply a hypothesis without proving it. Each was introduced as an optimisation and each quietly became a trust assumption. A caller passing a variable box is asserting that it contains D, and nothing downstream can detect a box that does not: every routine receiving one searches it honestly and reports a complete-looking answer about a feasible set that is not D.

All eleven were caught by asking, for each hypothesis stated in the text, which line of code establishes it. Eleven times the answer was none. None would have been caught by checking that the method gets the right answer on valid input, which is what the suite did for its first 93 tests.

106 tests in all, run on every commit, in exact rational arithmetic with no floating point anywhere and no third-party dependency. Ten of them feed deliberately invalid input and require a refusal.